Project Summary – Employee Salary Calculator

Project	Employee Salary Calculator
Problem Statement No.	PS No. 10 – B.Tech CSE
Language	Python 3 (single file: `salary_calculator.py`)
Type	Console (command-line) application
Concepts	Functions, Operators, Conditions, Loops, Input validation
Author	Amogh Agrawal

---

1. Introduction

In any organisation, an employee's salary is made up of several parts: a basic pay, allowances added on top, and deductions taken off. Working these out by hand for every employee is slow and mistakes are easy to make. This project automates that process with a simple Python program: given an employee's name, ID and basic salary, it calculates every component and prints a ready-to-read salary slip.

2. Problem Definition

Problem: Build a console program in Python 3 that calculates an employee's salary components and prints a salary slip.

Inputs: Employee name, employee ID, basic salary.

Processing: Compute HRA, DA, gross salary, PF, tax and net salary using arithmetic operators and conditional logic.

Output: A neat salary slip. The user can then process another employee, repeating until they choose to stop.

Constraints:
- Single `.py` file run from the console
- Must use functions (for example `calculate_gross()` and `print_slip()`)
- Must reject invalid salary input (negative or non-numeric)
- Must pass the four given test cases

3. Objectives

1. Understand the problem and define it clearly.
2. Develop the solution with proper logic and programming concepts.
3. Implement all required features and pass the given test cases.
4. Test and debug to make sure the output is correct.
5. Document the work and upload code and report to GitHub.

4. Salary Calculation Rules

Component	Rule
HRA (House Rent Allowance)	20% of basic salary
DA (Dearness Allowance)	10% of basic salary

Gross Salary	Basic + HRA + DA
PF (Provident Fund)	12% of basic salary
Tax	Based on gross: 0% up to 25,000; 5% up to 50,000; 10% above 50,000
Net Salary	Gross – PF – Tax

Key points about the rules:
- **Earnings** (HRA, DA) are added to basic to form **gross**.
- **Deductions** (PF, tax) are subtracted from gross to get **net** (take-home) pay.
- PF is calculated on **basic**, but tax is calculated on **gross**.
- Tax is a **flat rate** for the slab the gross falls in, not a progressive calculation.

5. Design and Approach

The program is split into small functions, each doing exactly one job. This keeps the code easy to read, test and change: if a rule changes (for example PF becomes 10%), only one function needs editing.

Function	Responsibility
`get_basic_salary()`	Reads salary and keeps asking until it is a valid positive number
`calculate_hra(basic)`	Returns 20% of basic
`calculate_da(basic)`	Returns 10% of basic
`calculate_gross(basic, hra, da)`	Returns basic + HRA + DA
`calculate_pf(basic)`	Returns 12% of basic
`calculate_tax(gross)`	Chooses the tax slab and returns the tax
`calculate_net(gross, pf, tax)`	Returns gross – PF – tax
`print_slip(...)`	Prints the formatted salary slip
`main()`	Reads name and ID, calls everything in order, loops for more employees

Order of execution inside `main()`:
`HRA → DA → Gross → PF → Tax → Net → Print slip`. This order is required because each step depends on the earlier ones (gross needs HRA and DA; tax needs gross; net needs gross, PF and tax).

6. How Each Programming Concept Is Used

Concept	Where and how it is used
Functions	Nine functions with parameters and return values; the calculation logic is separated from input and output.
Operators	Arithmetic operators (`*`, `/`, `+`, `-`) for every calculation; comparison operators (`<=`, `>`, `==`) for the tax slab, validation and loop control.
Conditions	`if / elif / else` in `calculate_tax()` selects the slab; `if` statements in `get_basic_salary()` check the input.
Loops	A `while True` loop in `get_basic_salary()` keeps asking until the input is valid; a `while choice == "y"` loop in `main()` allows many employees.
String methods	`replace()` and `isdigit()` validate the salary text; `lower()` lets the user answer `y` or `Y`.

7. Input Validation

A key part of the program is making sure bad input cannot crash it or produce wrong results. `get_basic_salary()` removes one decimal point from the text and checks that only digits remain, then converts to a number and confirms it is greater than zero.

Input	Outcome
`30000`, `25000.50`	Accepted
`-5000`	Rejected (minus sign is not a digit)
`abc`	Rejected (letters)
`0`	Rejected (not greater than zero)
empty input	Rejected

On any rejected input the program prints `Invalid salary. Enter a positive number.` and asks again, so it never crashes on a wrong entry.

8. Testing

Testing was done by running the program with the four test cases from the problem statement and comparing the output with the expected values. Screenshots of every run are in the `screenshots/` folder.

Test Case	Input	Expected Output	Actual Output	Status
1 – Normal input	30000	HRA 6000, DA 3000, Gross 39000, PF 3600, Tax 1950, Net 33450	Same	**Pass**
2 – Boundary / minimum	10000	HRA 2000, DA 1000, Gross 13000, PF 1200, Tax 0, Net 11800	Same	**Pass**
3 – Invalid input	-5000, then abc	`Invalid salary. Enter a positive number.`	Same (shown for both inputs, then re-prompts)	**Pass**
4 – Different valid	50000	HRA 10000, DA 5000, Gross 65000, PF 6000, Tax 6500, Net 52500	Same	**Pass**

What each test proves:
- **Test 1** confirms the normal calculation and the 5% tax slab (gross 39000).
- **Test 2** confirms the lowest slab gives zero tax (gross 13000).
- **Test 3** confirms that negative and non-numeric inputs are rejected and the program keeps asking instead of crashing.
- **Test 4** confirms the highest slab, 10% tax (gross 65000).

Together the valid cases exercise all three branches of the tax calculation (`if`, `elif`, `else`), and the invalid case exercises both failure paths of the validation.

Sample verification (Test 1):
HRA = 30000 × 20/100 = 6000; DA = 30000 × 10/100 = 3000; Gross = 30000 + 6000 + 3000 = 39000; PF = 30000 × 12/100 = 3600; Tax = 39000 × 5/100 = 1950; Net = 39000 − 3600 − 1950 = **33450**.

9. Sample Output

```
========================================
          SALARY SLIP
========================================
Employee Name : Amogh Agrawal
```

```
Employee ID   : E101
---------------------------------------
Basic Salary  : 30000
HRA (20%)     : 6000
DA (10%)      : 3000
Gross Salary  : 39000
---------------------------------------
PF (12%)      : 3600
Tax           : 1950
---------------------------------------
NET SALARY    : 33450
=======================================
HRA: 6000 | DA: 3000 | Gross: 39000
PF: 3600 | Tax: 1950 | Net Salary: 33450
```

10. Results and Outcome

The program meets all the bare-minimum requirements:

Requirement	Met
Python 3, single `.py` file, run from console	Yes
Takes name, ID and basic salary as input	Yes
Calculates HRA, DA, PF and tax using operators and conditions	Yes
Uses functions such as `calculate_gross()` and `print_slip()`	Yes
Prints a neat salary slip	Yes
Allows more employees using a loop	Yes
Passes all 4 given test cases	Yes

11. Limitations

- Tax is a flat slab rate, so crossing a slab boundary makes the tax jump (e.g. gross 25,000 has no tax but 25,001 is taxed about 1,250).
- Amounts on the slip are truncated to whole numbers with `int()`, not rounded.
- Employee name and ID are not validated and can be left empty.
- Slips are only displayed, not saved to a file.
- Numbers with spaces or commas (e.g. `" 5000"`, `10,000`) are rejected.

12. Future Scope

- Validate name and ID, and trim extra spaces in all inputs.
- Show amounts with two decimal places.
- Save each salary slip to a text or CSV file.
- Show a combined report for all employees processed in one session.
- Use progressive (marginal) tax slabs.
- Add unit tests for the calculation functions.
- Build a GUI or web version.

13. Conclusion

The Employee Salary Calculator is a complete, working solution to PS No. 10. It applies functions, operators, conditions and loops to a realistic payroll problem, handles invalid input

safely, and produces output that matches all four given test cases exactly. Splitting the logic into small functions makes the program easy to understand, test and extend.

TEST CASE SCREENSHOTS

TAP HERE TO SEE SCREENSHOTS- https://drive.google.com/file/d/1SUHmNfK7f7bLuo6kocGzGw_QuM26HMgu/view?usp=sharing